

Book Store QA

Testing manual — unit, API and UI

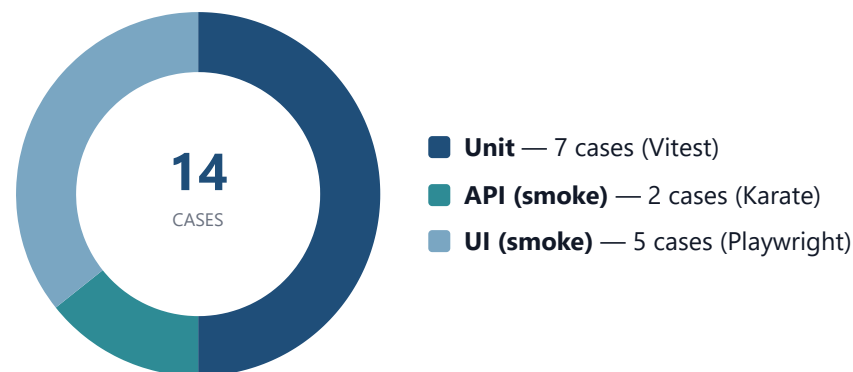
Generated 28 August 2026, 13:33 · commit 7bea172
Vitest · Karate · Playwright · GitHub Actions

Executive test metrics

Every figure below was produced by executing the suites at commit `479aa3d`, not by counting source. Regenerate with `python scripts/collect_test_metrics.py`.



Smoke gate by layer



Test suite / layer	Tool	Total	Passed	Failed	Result	Coverage / scope
Unit	Vitest	7	7	0	100%	Pure logic in src/ - 100% of the covered surface
API (smoke)	Karate	2	2	0	100%	@smoke scenarios - the deployment gate
UI (smoke)	Playwright	5	5	0	100%	@smoke journeys on chromium
API (regression)	Karate	23	23	0	100%	Every scenario, all tags
UI (regression)	Playwright	7	7	0	100%	Every spec on chromium

The smoke gate is the first three rows: what a pull request runs and what verifies a production deployment. The regression rows are shown apart from it deliberately — quoting the API layer as 2 cases without saying *smoke* would misrepresent a suite that has 23.

1. Testing architecture

Three layers, run in a strict sequence. Each is cheaper and more certain than the one after it, so the pipeline learns the cheapest thing first and stops as soon as it knows enough.

Layer	Tool	Scope	Needs a target URL?	Fails the stage below it
Unit	Vitest	Pure logic in <code>playwright/src</code> — no browser, no network	No	Skips API and UI
API	Karate (Java 17 + Maven)	Book Store REST contract, per scenario	Yes	Skips UI
UI	Playwright	Browser journeys, one leg per browser	Yes	—

```
0. unit (Vitest) → api (Karate) → ui (Playwright, one leg per browser)
```

Why unit tests run once, not per environment. Unit tests assert pure logic — they never open a browser, never make a request, never touch `demoqa.com`. Their answer therefore *cannot* differ between dev, release and production. Running them inside each stage bought three identical answers to the same question, and made the API suite wait behind a redundant `npm ci` every time. They now run once, straight after the run configuration is resolved, as job `0. Unit tests (Vitest)`.

Because they run once, they have to gate *every* way into the chain. `dev` inherits that through ordinary job dependencies, but `release` and `production` are entry points in their own right and their conditions use a status function, which opts out of the implicit "all dependencies succeeded" rule. Both therefore name the result explicitly — without that, starting a run from `release` would walk straight past the gate.

2. Local execution

Three parameters run through everything: **environment** (which host), **suite** (smoke or regression) and **browser**.

Windows PowerShell. Run the lines in each block one at a time. PowerShell 5.1 has no `&&`, so joining them fails with `"'&&' is not a valid statement separator"`. Use `;` to chain, or `; if ($?) { ... }` to stop on the first failure. Git Bash and PowerShell 7+ accept `&&` as written. PowerShell also has no inline `VAR=value` command prefix — use `$env:VAR='...'` on its own statement.

Quote Playwright's tags on PowerShell too. `@` is the splatting operator, so a bare `--grep @smoke` expands the undefined variable `$smoke` to nothing and the argument silently disappears — Playwright then reports `"option '-g, --grep' argument missing"`. Write `--grep '@smoke'`.

Unit tests and coverage (Vitest)

```
cd playwright
npm ci
npm run test:unit           # tests only
npm run test:unit -- --coverage # tests + coverage gate
npm run test:unit:watch     # re-run on save while developing
```

Coverage is scoped to the code unit tests are responsible for. `src/pages/**` and `src/utils/api-client.ts` are excluded by rule, because the Playwright suite and the live API are what exercise them — counting them

here would measure the absence of Playwright rather than the quality of these tests.

Metric	Covered	Total	%	Floor
Statements	2	2	100%	80%
Branches	1	1	100%	80%
Functions	1	1	100%	80%
Lines	2	2	100%	80%

Vitest fails the run itself when any metric drops below the floor, so the gate is the runner's and not a later script's reading of a report. Read the figure as "*the pure logic is covered*", not "*the project is covered*" — the covered surface is deliberately small.

```
playwright - npm

$ npm run test:unit -- --coverage

> bookstore-web-automation@1.0.0 test:unit
> vitest run --coverage
RUN v4.1.11 C:/Users/g6k78/qa-assessment/playwright
  Coverage enabled with v8
✓ src/utils/test-data.test.ts (7 tests) 18ms
Test Files  1 passed (1)
Tests       7 passed (7)
Start at   13:56:39
Duration   469ms (transform 66ms, setup 0ms, import 105ms, tests 18ms, environment 0ms)
JUNIT report written to C:/Users/g6k78/qa-assessment/playwright/test-results/unit-junit.xml
% Coverage report from v8
===== Coverage summary =====
Statements : 100% ( 2/2 )
Branches   : 100% ( 1/1 )
Functions  : 100% ( 1/1 )
Lines      : 100% ( 2/2 )
=====
exit 0 - passed · 1.9s
```

API tests (Karate)

Requires **Java 17** and Maven 3.8+. Karate's GraalJS engine does not support JDK 18+ — on a newer JDK the suite hangs silently instead of failing, so check `java -version` first.

```
cd karate
```

Quote any `-D` containing a dot on PowerShell: it ends a parameter name at the first `.`, so `-Dkarate.env=dev` arrives at Maven split in two and fails with "*Unknown lifecycle phase '.env=dev'*".

Dev — smoke:

```
mvn test -Dtest=SmokeTest -Dkarate.env=dev
```

```
mvn test '-Dtest=SmokeTest' '-Dkarate.env=dev'
```

```
karate - mvn

$ mvn test -Dtest=SmokeTest -Dkarate.env=dev

[INFO] Scanning for projects...
[INFO] -----< com.assessment.qa:bookstore-api-automation >-----
[INFO] Building Book Store API Automation (Karate) 1.0.0
[INFO]   from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- resources:3.4.0:resources (default-resources) @ bookstore-api-automation ---
[INFO] skip non existing resourceDirectory C:\Users\g6k78\qa-assessment\karate\src\main\resources
[INFO] --- compiler:3.15.0:compile (default-compile) @ bookstore-api-automation ---
[INFO] No sources to compile
[INFO] --- resources:3.4.0:testResources (default-testResources) @ bookstore-api-automation ---
[INFO] Copying 8 resources from src\test\java to target\test-classes
... 140 lines omitted ...
feature: classpath:bookstore/account/account.feature
scenarios: 1 | passed: 1 | failed: 0 | time: 9,2930
-----
13:57:07.309 [pool-3-thread-1] INFO  c.i.k.Suite - <<pass>> feature 1 of 5 (4 remaining) classpath:bookstore/account/account.feature
-----
feature: classpath:bookstore/books/books.feature
scenarios: 1 | passed: 1 | failed: 0 | time: 5,4620
-----
13:57:07.788 [pool-3-thread-1] INFO  c.i.k.Suite - <<pass>> feature 2 of 5 (3 remaining) classpath:bookstore/books/books.feature
Karate version: 1.4.1 | env: dev
-----
elapsed: 14,86 | threads: 3 | thread time: 14,76
features: 2 | skipped: 3 | efficiency: 0,33
scenarios: 2 | passed: 2 | failed: 0
-----
HTML report: (paste into browser to view) | Karate version: 1.4.1 | env: dev
file:///C:/Users/g6k78/qa-assessment/karate/target/karate-reports/karate-summary.html
-----
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 15.87 s -- in bookstore.SmokeTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 22.314 s
[INFO] Finished at: 2026-08-27T13:57:08+03:00
[INFO] -----

exit 0 - passed - 26.1s
```

Release — full regression:

```
mvn test -Dtest=BookStoreApiTest -Dkarate.env=release -DbaseUrl=https://demoqa.com
```

```
mvn test '-Dtest=BookStoreApiTest' '-Dkarate.env=release' '-DbaseUrl=https://demoqa.com'
```

```
karate - mvn

$ mvn test -Dtest=BookStoreApiTest -Dkarate.env=release -DbaseUrl=https://demoqa.com

[INFO] Scanning for projects...
[INFO] -----< com.assessment.qa:bookstore-api-automation >-----
[INFO] Building Book Store API Automation (Karate) 1.0.0
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- resources:3.4.0:resources (default-resources) @ bookstore-api-automation ---
[INFO] skip non existing resourceDirectory C:\Users\g6k78\qa-assessment\karate\src\main\resources
[INFO] --- compiler:3.15.0:compile (default-compile) @ bookstore-api-automation ---
[INFO] No sources to compile
[INFO] --- resources:3.4.0:testResources (default-testResources) @ bookstore-api-automation ---
[INFO] Copying 8 resources from src\test\java to target\test-classes
... 1617 lines omitted ...
9 < Connection: keep-alive
9 < X-Powered-By: Express
9 < ETag: W/"2b-hLPYyb0m8IpHH/aFjmp8VZrYXGw"
{"code":"1207","message":"User not found!"}
-----
feature: classpath:bookstore/e2e/bookstore-collection.feature
scenarios: 1 | passed: 1 | failed: 0 | time: 16,0475
-----
13:58:10.083 [pool-3-thread-1] INFO c.i.k.Suite - <<pass>> feature 5 of 5 (0 remaining) classpath:bookstore/e2e/bookstore-collection.feature
Karate version: 1.4.1 | env: release
=====
elapsed: 51,07 | threads: 5 | thread time: 193,43
features: 3 | skipped: 2 | efficiency: 0,76
scenarios: 23 | passed: 23 | failed: 0
=====
HTML report: (paste into browser to view) | Karate version: 1.4.1 | env: release
file:///C:/Users/g6k78/qa-assessment/karate/target/karate-reports/karate-summary.html
=====
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 51.93 s -- in bookstore.BookStoreApiTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 56.159 s
[INFO] Finished at: 2026-08-27T13:58:10+03:00
[INFO] -----

exit 0 - passed · 59.7s
```

Production — smoke, with an explicit host:

```
mvn test -Dtest=SmokeTest -Dkarate.env=production -DbaseUrl=https://demoqa.com
```

```
mvn test '-Dtest=SmokeTest' '-Dkarate.env=production' '-DbaseUrl=https://demoqa.com'
```

```
karate - mvn

$ mvn test -Dtest=SmokeTest -Dkarate.env=production -DbaseUrl=https://demoqa.com

[INFO] Scanning for projects...
[INFO] -----< com.assessment.qa:bookstore-api-automation >-----
[INFO] Building Book Store API Automation (Karate) 1.0.0
[INFO] from pom.xml
[INFO] -----[ jar ]-----
[INFO] --- resources:3.4.0:resources (default-resources) @ bookstore-api-automation ---
[INFO] skip non existing resourceDirectory C:\Users\g6k78\qa-assessment\karate\src\main\resources
[INFO] --- compiler:3.15.0:compile (default-compile) @ bookstore-api-automation ---
[INFO] No sources to compile
[INFO] --- resources:3.4.0:testResources (default-testResources) @ bookstore-api-automation ---
[INFO] Copying 8 resources from src\test\java to target\test-classes
... 140 lines omitted ...
feature: classpath:bookstore/account/account.feature
scenarios: 1 | passed: 1 | failed: 0 | time: 5,7354
-----
13:58:30.105 [pool-3-thread-1] INFO c.i.k.Suite - <<pass>> feature 1 of 5 (4 remaining) classpath:bookstore/account/account.feature
-----
feature: classpath:bookstore/books/books.feature
scenarios: 1 | passed: 1 | failed: 0 | time: 5,6522
-----
13:58:30.549 [pool-3-thread-1] INFO c.i.k.Suite - <<pass>> feature 2 of 5 (3 remaining) classpath:bookstore/books/books.feature
Karate version: 1.4.1 | env: production
=====
elapsed: 10,24 | threads: 3 | thread time: 11,39
features: 2 | skipped: 3 | efficiency: 0,37
scenarios: 2 | passed: 2 | failed: 0
=====
HTML report: (paste into browser to view) | Karate version: 1.4.1 | env: production
file:///C:/Users/g6k78/qa-assessment/karate/target/karate-reports/karate-summary.html
=====
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 11.07 s -- in bookstore.SmokeTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 15.185 s
[INFO] Finished at: 2026-08-27T13:58:31+03:00
[INFO] -----

exit 0 - passed - 18.6s
```

`-Dtest` picks the runner class and therefore the tag set: `SmokeTest` runs `@smoke`, `BookStoreApiTest` runs everything.

`release` and `production` both need `-DbaseUrl` locally. `dev` defaults to the demo site, but `release` falls back to a placeholder host that does not resolve, so the command would fail with a connection error rather than a test failure. In CI the host comes from the `RELEASE_BASE_URL` repository variable instead. Reports land in `karate/target/karate-reports/karate-summary.html`.

UI tests (Playwright)

Run every command in this section from `playwright/`, not from the repository root. Playwright is a dev dependency of that package, and its config lives there.

Two symptoms tell you that you are in the wrong directory. `npx` offers to download a throwaway copy — *"Need to install the following packages: playwright@... Ok to proceed?"* — and, having found no config, it then reports

```
Error: Project(s) "chromium" not found. Available projects: ""
```

An empty *available projects* list always means the config was not found, never that the project is misspelled. `cd playwright` and run it again.

```
cd playwright
npx playwright install chromium          # or: firefox webkit msedge
```

```
Set-Location playwright
npx playwright install chromium
```

Dev — smoke on chromium:

```
BASE_URL=https://demoqa.com npx playwright test --project=chromium --grep @smoke
```

```
$env:BASE_URL='https://demoqa.com'; npx playwright test --project=chromium --grep '@smoke'
```

```
playwright - npx

$ BASE_URL=https://demoqa.com npx playwright test --project=chromium --grep @smoke

Running 5 tests using 4 workers
ok 3 [chromium] > tests\login.spec.ts:27:7 > Login @smoke > protects the profile page from anonymous access (6.9s)
ok 5 [chromium] > tests\registration.spec.ts:32:7 > Registration @smoke > the API rejects a password that violates the documented policy (279ms)
ok 4 [chromium] > tests\registration.spec.ts:11:7 > Registration @smoke > the registration form is reachable from login and is captcha-protected (7.2s)
ok 1 [chromium] > tests\login.spec.ts:4:7 > Login @smoke > rejects unknown credentials with a visible error and no session (8.7s)
ok 2 [chromium] > tests\login.spec.ts:19:7 > Login @smoke > rejects a valid username with the wrong password (10.9s)
5 passed (12.4s)

exit 0 - passed · 15.0s
```

Release — regression across every browser:

```
BASE_URL=https://demoqa.com npx playwright test --project=chromium --project=firefox --
project=webkit --project=msedge
```

```
$env:BASE_URL='https://demoqa.com'; npx playwright test --project=chromium --project=firefox --
project=webkit --project=msedge
```



```
playwright - npx

$ BASE_URL=https://demoqa.com npx playwright test --project=chromium --project=firefox --project=webkit --project=msedge

Running 28 tests using 4 workers
  ok 4 [chromium] > tests\login.spec.ts:4:7 > Login @smoke > rejects unknown credentials with a visible error and no session (7.8s)
  ok 5 [chromium] > tests\login.spec.ts:27:7 > Login @smoke > protects the profile page from anonymous access (1.5s)
  ok 6 [chromium] > tests\registration.spec.ts:11:7 > Registration @smoke > the registration form is reachable from login and is captcha-protected (2.4s)
  ok 7 [chromium] > tests\registration.spec.ts:32:7 > Registration @smoke > the API rejects a password that violates the documented policy (282ms)
  ok 2 [chromium] > tests\login.spec.ts:19:7 > Login @smoke > rejects a valid username with the wrong password (14.9s)
  ok 3 [chromium] > tests\bookstore-collection.spec.ts:80:7 > Book Store - my collection @e2e > cancelling the delete dialog keeps the book in the collection @regression (18.7s)
  ok 1 [chromium] > tests\bookstore-collection.spec.ts:15:7 > Book Store - my collection @e2e > a user can register, add a book to their collection, remove it and log out (21.1s)
  ok 10 [firefox] > tests\login.spec.ts:4:7 > Login @smoke > rejects unknown credentials with a visible error and no session (16.2s)
  ok 12 [firefox] > tests\login.spec.ts:27:7 > Login @smoke > protects the profile page from anonymous access (3.6s)
  ok 11 [firefox] > tests\login.spec.ts:19:7 > Login @smoke > rejects a valid username with the wrong password (18.6s)
  ok 14 [firefox] > tests\registration.spec.ts:32:7 > Registration @smoke > the API rejects a password that violates the documented policy (155ms)
  ok 9 [firefox] > tests\bookstore-collection.spec.ts:80:7 > Book Store - my collection @e2e > cancelling the delete dialog keeps the book in the collection @regression (25.1s)
  ok 13 [firefox] > tests\registration.spec.ts:11:7 > Registration @smoke > the registration form is reachable from login and is captcha-protected (4.5s)
  ok 8 [firefox] > tests\bookstore-collection.spec.ts:15:7 > Book Store - my collection @e2e > a user can register, add a book to their collection, remove it and log out (36.7s)
  ok 17 [webkit] > tests\login.spec.ts:4:7 > Login @smoke > rejects unknown credentials with a visible error and no session (8.2s)
  ok 19 [webkit] > tests\login.spec.ts:27:7 > Login @smoke > protects the profile page from anonymous access (3.5s)
  ok 20 [webkit] > tests\registration.spec.ts:11:7 > Registration @smoke > the registration form is reachable from login and is captcha-protected (3.9s)
  ok 21 [webkit] > tests\registration.spec.ts:32:7 > Registration @smoke > the API rejects a password that violates the documented policy (281ms)
  ok 18 [webkit] > tests\login.spec.ts:19:7 > Login @smoke > rejects a valid username with the wrong password (14.1s)
  ok 15 [webkit] > tests\bookstore-collection.spec.ts:15:7 > Book Store - my collection @e2e > a user can register, add a book to their collection, remove it and log out (24.3s)
  ok 16 [webkit] > tests\bookstore-collection.spec.ts:80:7 > Book Store - my collection @e2e > cancelling the delete dialog keeps the book in the collection @regression (23.8s)
  ok 24 [msedge] > tests\login.spec.ts:4:7 > Login @smoke > rejects unknown credentials with a visible error and no session (9.9s)
  ok 25 [msedge] > tests\login.spec.ts:19:7 > Login @smoke > rejects a valid username with the wrong password (12.7s)
  ok 26 [msedge] > tests\login.spec.ts:27:7 > Login @smoke > protects the profile page from anonymous access (3.7s)
  ok 28 [msedge] > tests\registration.spec.ts:32:7 > Registration @smoke > the API rejects a password that violates the documented policy (366ms)
  ok 23 [msedge] > tests\bookstore-collection.spec.ts:80:7 > Book Store - my collection @e2e > cancelling the delete dialog keeps the book in the collection @regression (19.2s)
```

Production — smoke on Edge:

```
BASE_URL=https://demoqa.com npx playwright test --project=msedge --grep @smoke
```

```
$env:BASE_URL='https://demoqa.com'; npx playwright test --project=msedge --grep '@smoke'
```

```
playwright - npx

$ BASE_URL=https://demoqa.com npx playwright test --project=msedge --grep @smoke

Running 5 tests using 4 workers
  ok 4 [msedge] > tests\registration.spec.ts:11:7 > Registration @smoke > the registration form is reachable from login and is captcha-protected (12.5s)
  ok 1 [msedge] > tests\login.spec.ts:27:7 > Login @smoke > protects the profile page from anonymous access (12.7s)
  ok 5 [msedge] > tests\registration.spec.ts:32:7 > Registration @smoke > the API rejects a password that violates the documented policy (690ms)
  ok 2 [msedge] > tests\login.spec.ts:4:7 > Login @smoke > rejects unknown credentials with a visible error and no session (14.0s)
  ok 3 [msedge] > tests\login.spec.ts:19:7 > Login @smoke > rejects a valid username with the wrong password (10.7s)
  5 passed (17.9s)

exit 0 - passed · 21.0s
```

UI mode — pick and re-run tests interactively:

```
npx playwright test --ui
npm run test:headed      # or just watch chromium drive
npm run report            # open the last HTML report
```

Edge is a branded channel rather than a bundled engine, so `npx playwright install` alone does not fetch it — it has to be named. `playwright test` has no `--channel` flag either, so the config models Edge as a project pinning channel: `'msedge'`; one `--project` argument then selects any of the four.

The whole sequence in one line

Unit → API → UI, stopping at the first failure:

```
cd playwright && npm run test:unit -- --coverage && cd ../karate && mvn test -Dtest=SmokeTest -
Dkarate.env=dev && cd ../playwright && BASE_URL=https://demoqa.com npx playwright test --
project=chromium --grep @smoke
```

```
cd playwright; if ($?) { npm run test:unit -- --coverage }; if ($?) { cd ../karate }; if ($?) {
mvn test '-Dtest=SmokeTest' '-Dkarate.env=dev' }; if ($?) { cd ../playwright }; if ($?) {
$env:BASE_URL='https://demoqa.com'; npx playwright test --project=chromium --grep '@smoke' }
```

3. GitHub Actions pipeline

Run it from **Actions** → **QA Automation** → **Run workflow**. Four things decide what happens, and the first of them is the branch:

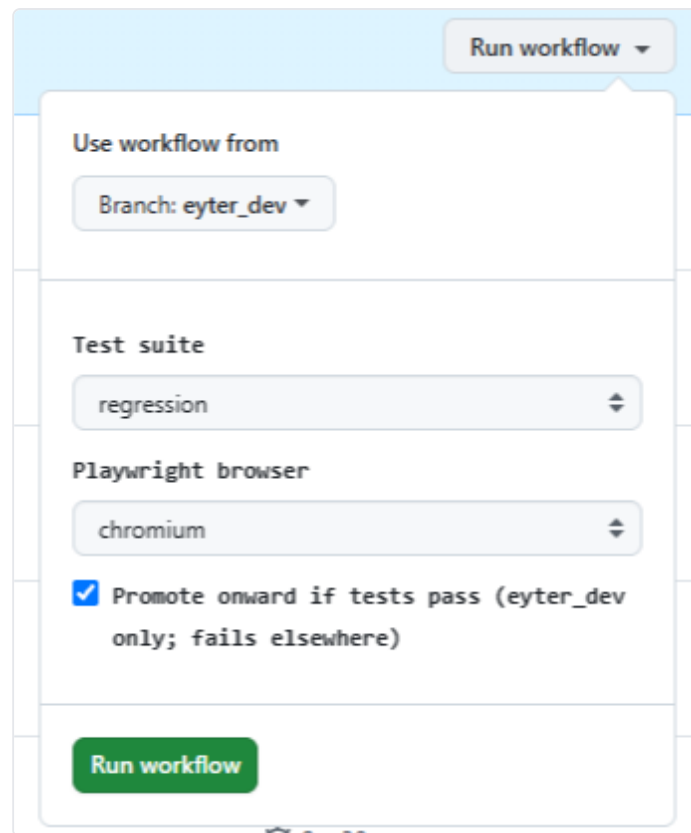
Input	Options	Effect
Use workflow from	<code>eyter_dev</code> , <code>release</code> , <code>main</code>	Chooses the entry stage — this is why there is no environment dropdown
Test suite	<code>smoke</code> , <code>regression</code>	<code>-Dtest=SmokeTest / BookStoreApiTest</code> , and <code>--grep @smoke</code>
Playwright browser	<code>chromium</code> , <code>firefox</code> , <code>webkit</code> , <code>msedge</code> , <code>all browsers</code>	One matrix leg each; <code>all browsers</code> runs four in parallel
Promote	checkbox, off by default	Whether the run continues past its entry stage

Scenario A — running from `eyter_dev`

The full chain. Unit tests gate everything; the dev stage runs against the dev host; if it is green and *Promote* is ticked, the tested commit is fast-forwarded onto `release`, the release stage runs, and the same happens onto `main` before production is deployed and verified.

```
0. unit → 1. dev → promote → 2. release → promote → 3. deploy → verify
```

With *Promote* unticked the run stops after the dev stage: nothing is promoted, no branch moves, nothing is deployed.



The image shows a modal window for running a workflow. At the top right is a button labeled "Run workflow" with a dropdown arrow. Below this is a section titled "Use workflow from" containing a dropdown menu showing "Branch: eyter_dev". The next section is titled "Test suite" and contains a dropdown menu showing "regression". Below that is a section titled "Playwright browser" with a dropdown menu showing "chromium". At the bottom of this section is a checkbox labeled "Promote onward if tests pass (eyter_dev only; fails elsewhere)" which is checked. At the very bottom of the modal is a green button labeled "Run workflow".

Scenario B — running from `release` or `main`

The branch is the entry point, so earlier stages are skipped and the run starts where you pointed it. **Promotion is bound to `eyter_dev`**: ticking *Promote* on a run started from `release`, `main` or a feature branch fails the run in seconds, before any suite starts, with

Promotion is only allowed when triggered from the `eyter_dev` branch.

That guard matters most in the case that looks harmless — a feature branch enters at the dev stage exactly like `eyter_dev` does, so without it a green feature-branch run would push its own commit straight at `release`.

Two things to know before running from `main`: entering at stage 3 **deploys**, promote or not, because `main` has nowhere further to promote to; and a `Pipeline result` job fails the run if production was deployed but its verification did not pass, so a deploy can never report green unverified.

QA Automation

✖ [MANUAL: release] - Promote onward from this branch if its tests pass - smoke on chromium #68

Re-run jobs

Summary

All jobs

✖ Resolve run configuration

1. Dev

1b. Promote to release

2. Release

2b. Promote to main

3. Deploy to production

3b. Production

Pipeline result

Run details

Usage

Workflow file

Manually triggered yesterday

Eyter-Higuera

ed4d679

release

Status

Failure

Total duration

8s

Artifacts

—

ci.yml

on: workflow_dispatch

Resolve run configuration

1. Dev / opt

Waiting for pending jobs

1b. Promote to release

1. Release / opt

Waiting for pending jobs

1b. Promote to main

1. Deploy to production

production-verify / opt

Waiting for pending jobs

Pipeline result

Annotations

2 errors

✖ Resolve run configuration

Process completed with exit code 1.

✖ Promotion not allowed from this branch

Promotion is only allowed when triggered from the eyter_dev branch.

What the run summary shows

Every job writes its results to the run summary page, so a failure is readable without downloading an artifact: the Vitest coverage table with a tick or cross per metric, then a Karate table and a Playwright table per stage — totals, a row per suite with pass/fail/skip counts and durations, and, when something fails, a second table naming each failing test with the first line of its error.

Book Store QA — testing manual

12

/

21

Summary

All jobs

Resolve run configuration

0. Unit tests (Vitest)

1. Dev

1b. Promote to release

2. Release

2b. Promote to main

3. Deploy to production

3b. Production smoke

Pipeline result

Run details

Usage

Workflow file

Resolve run configuration summary

Run configuration

Suites: [API + UI] - Karate then Playwright at every stage.

Commit: 46acaf4

Triggered by workflow_dispatch on eytar_dev, entering at dev.

The dev -> release -> production chain runs only from eytar_dev. A manual run from release or main tests that stage and stops; ticking promote there fails the run before any suite starts.

Promotion is off: this run tests its entry stage and stops.

Stage	Runs
1. dev	entry point - smoke on ["chromium"] - https://demoqa.com
2. release	skipped - promotion is off
3. production	skipped - promotion is off

Job summary generated at run-time

0. Unit tests (Vitest) summary

Unit (Vitest)

passed - 7 tests in 0.0s: 7 passed, 0 failed, 0 skipped

Suite	Passed	Failed	Skipped	Duration
src/utills/test-data.test.ts	7	0	0	0.0s

Unit test coverage

Metric	Covered	Total	%	Threshold 80%
Statements	2	2	100.0%	✓
Branches	1	1	100.0%	✓
Functions	1	1	100.0%	✓
Lines	2	2	100.0%	✓

► Per file (1)

Job summary generated at run-time

1. Dev / API (Karate) - dev - smoke summary

Karate - dev - smoke

passed - 2 tests in 17.2s: 2 passed, 0 failed, 0 skipped

Suite	Passed	Failed	Skipped	Duration
bookstore/account/account.feature	1	0	0	10.1s
bookstore/books/books.feature	1	0	0	7.1s

Job summary generated at run-time

1. Dev / Web (Playwright) - dev - smoke - chromium summary

Playwright - dev - smoke - chromium

passed - 5 tests in 14.6s: 5 passed, 0 failed, 0 skipped

Suite	Passed	Failed	Skipped	Duration
login.spec.ts	3	0	0	12.6s
registration.spec.ts	2	0	0	2.1s

4. Local pipeline simulation & multi-environment matrix guide

The pipeline can be run end to end on a laptop, with the same stages, the same order and the same parameters as CI. One entry point covers the whole matrix:

```
node scripts/pipeline.mjs --stage <dev|release|production> \
    --suite <unit|api|ui|smoke|regression> \
    --browser <chromium|firefox|webkit|msedge|all>
```

```
node scripts/pipeline.mjs --stage dev --suite smoke --browser chromium
```

The same line works in both shells, and from any directory in the repository. That is deliberate — see *Why a runner rather than raw commands* below.

```

$ node scripts/pipeline.mjs --stage release --suite regression --browser all --dry-run

Local pipeline
  stage   release (BASE_URL https://demoqa.com - from default)
  suite   regression → unit → api → ui
  browser chromium, firefox, webkit, msedge
[dry run] unit: node C:\Users\g6k78\qa-assessment\playwright\node_modules\vitest\vitest.mjs run --coverage
[dry run] api: mvn -B test -Dtest=BookStoreApiTest -Dkarate.env=release -DbaseUrl=https://demoqa.com
[dry run] ui: node C:\Users\g6k78\qa-assessment\playwright\node_modules\playwright\cli.js test --project=chromium --project=firefox --project=webkit --project=msedge
Summary
  unit  passed  0.0s
  api   passed  0.0s
  ui    passed  0.0s
exit 0 - passed - 0.2s

```

The three parameters

Parameter	Values	What it changes
<code>--stage</code>	dev, release, production	BASE_URL for Playwright and <code>-Dkarate.env</code> / <code>-DbaseUrl</code> for Karate
<code>--suite</code>	unit, api, ui, smoke, regression	Which layers run, and how deep
<code>--browser</code>	chromium, firefox, webkit, msedge, all	One Playwright project each; <code>all</code> runs the four together

`unit`, `api` and `ui` run that layer **alone and in full**. `smoke` and `regression` run **all three layers** in the CI order — shallow or deep:

```
unit (Vitest) → api (Karate) → ui (Playwright)
```

A failing layer stops the run and the later ones are reported as skipped, exactly as the workflow does. Add `--dry-run` to print the resolved commands without executing anything — the quickest way to check what a combination will do.

Where BASE_URL comes from

Hosts are read from the environment first and fall back to the public demo site, mirroring the repository variables CI uses. The runner prints which source it used, so a run never leaves you guessing what it tested:

Stage	Environment variable	Fallback
dev	DEV_BASE_URL , then STAGING_BASE_URL	https://demoqa.com
release	RELEASE_BASE_URL	https://demoqa.com
production	PRODUCTION_BASE_URL	https://demoqa.com

```
DEV_BASE_URL=https://dev.internal node scripts/pipeline.mjs --stage dev --suite smoke
```

```
$env:DEV_BASE_URL='https://dev.internal'; node scripts/pipeline.mjs --stage dev --suite smoke
```

npm shortcuts

Run these from the repository root. They are thin wrappers over the same script, so anything below can also be spelled out with explicit flags:

```
npm run dev:smoke           # unit → api → ui, smoke, chromium
npm run dev:regression      # the same, full depth
npm run dev:regression:all  # full depth on all four browsers
npm run release:smoke
npm run release:regression  # full depth, all browsers
npm run production:smoke
npm run test:unit           # one layer only
npm run test:api
npm run test:ui
npm run pipeline -- --stage release --suite ui --browser webkit
npm run pipeline:help
```

The `--` in the last example matters: without it npm keeps the flags for itself rather than passing them on.

The matrix, and what each combination is for

Combination	Command	Use
Dev smoke, chromium	<code>npm run dev:smoke</code>	The pull-request gate. Fastest useful signal
Dev regression, firefox	<code>node scripts/pipeline.mjs --stage dev --suite regression --browser firefox</code>	Engine-specific check before promoting
Dev regression, all	<code>npm run dev:regression:all</code>	What the release stage runs in CI
Release regression, all	<code>npm run release:regression</code>	Full pre-production sweep
Production smoke, webkit	<code>node scripts/pipeline.mjs --stage production --suite smoke --browser webkit</code>	Post-deployment verification
Unit only	<code>npm run test:unit</code>	Sub-second feedback while editing helpers
API only	<code>npm run test:api</code>	Contract work without a browser

npm

```
$ npm run dev:smoke
```

```
> qa-assessment@1.0.0 dev:smoke
```

```
> node scripts/pipeline.mjs --stage dev --suite smoke --browser chromium
```

```
Local pipeline
  stage    dev    (BASE_URL https://demoqa.com -- from default)
  suite    smoke  → unit → api → ui
  browser  chromium
$ node C:\Users\g6k78\qa-assessment\playwright\node_modules\vitest\vitest.mjs run --coverage
(in .\playwright)
RUN v4.1.11 C:/Users/g6k78/qa-assessment/playwright
  Coverage enabled with v8
✓ src/utils/test-data.test.ts (7 tests) 9ms
Test Files  1 passed (1)
Tests       7 passed (7)
Start at   11:06:14
Duration   394ms (transform 43ms, setup 0ms, import 70ms, tests 9ms, environment 0ms)
JUNIT report written to C:/Users/g6k78/qa-assessment/playwright/test-results/unit-junit.xml
% Coverage report from v8
... 172 lines omitted ...
Karate version: 1.4.1 | env: dev
```

```
=====
elapsed: 10,53 | threads: 3 | thread time: 9,74
features: 2 | skipped: 3 | efficiency: 0,31
scenarios: 2 | passed: 2 | failed: 0
=====
```

```
HTML report: (paste into browser to view) | Karate version: 1.4.1 | env: dev
file:///C:/Users/g6k78/qa-assessment/karate/target/karate-reports/karate-summary.html
=====
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 11.45 s -- in bookstore.SmokeTest
```

```
[INFO]
```

```
[INFO] Results:
```

```
[INFO]
```

```
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
```

```
[INFO]
```

```
[INFO] -----
```

```
[INFO] BUILD SUCCESS
```

```
[INFO] -----
```

```
[INFO] Total time: 16.013 s
```

```
[INFO] Finished at: 2026-08-28T11:06:34+03:00
```

```
[INFO] -----
```

```
$ node C:\Users\g6k78\qa-assessment\playwright\node_modules\playwright\cli.js test --project=chromium --grep @smoke
(in .\playwright)
```

```
Running 5 tests using 4 workers
```

```
ok 1 [chromium] > tests\login.spec.ts:27:7 > Login @smoke > protects the profile page from anonymous access (4.8s)
```

```
ok 5 [chromium] > tests\registration.spec.ts:32:7 > Registration @smoke > the API rejects a password that violates the documented policy (312ms)
```

```
ok 4 [chromium] > tests\registration.spec.ts:11:7 > Registration @smoke > the registration form is reachable from login and is captcha-protected (5.6s)
```

```
ok 2 [chromium] > tests\login.spec.ts:4:7 > Login @smoke > rejects unknown credentials with a visible error and no session (7.6s)
```

```
ok 3 [chromium] > tests\login.spec.ts:19:7 > Login @smoke > rejects a valid username with the wrong password (9.9s)
```

```
5 passed (11.4s)
```

```
Summary
```

```
unit passed 1.5s
```

```
api passed 18.9s
```

```
ui passed 12.3s
```

```

. - node

$ node scripts/pipeline.mjs --stage dev --suite regression --browser firefox

Local pipeline
  stage    dev (BASE URL https://demoqa.com - from default)
  suite    regression → unit → api → ui
  browser  firefox
$ node C:\Users\g6k78\qa-assessment\playwright\node_modules\vitest\vitest.mjs run --coverage
(in .\playwright)
RUN v4.1.11 C:/Users/g6k78/qa-assessment/playwright
  Coverage enabled with v8
✓ src/utils/test-data.test.ts (7 tests) 11ms
Test Files  1 passed (1)
Tests       7 passed (7)
Start at    11:06:49
Duration    447ms (transform 45ms, setup 0ms, import 72ms, tests 11ms, environment 0ms)
JUNIT report written to C:/Users/g6k78/qa-assessment/playwright/test-results/unit-junit.xml
% Coverage report from v8
===== Coverage summary =====
Statements   : 100% ( 2/2 )
... 1649 lines omitted ...
elapsed: 48,22 | threads: 5 | thread time: 182,01
features: 3 | skipped: 2 | efficiency: 0,75
scenarios: 23 | passed: 23 | failed: 0
=====
HTML report: (paste into browser to view) | Karate version: 1.4.1 | env: dev
file:///C:/Users/g6k78/qa-assessment/karate/target/karate-reports/karate-summary.html
=====
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 49.29 s -- in bookstore.BookStoreApiTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 53.747 s
[INFO] Finished at: 2026-08-28T11:07:46+03:00
[INFO] -----
$ node C:\Users\g6k78\qa-assessment\playwright\node_modules\playwright\cli.js test --project=firefox
(in .\playwright)
Running 7 tests using 4 workers
  ok 2 [firefox] > tests\login.spec.ts:4:7 > Login @smoke > rejects unknown credentials with a visible error and no session (24.3s)
  ok 1 [firefox] > tests\login.spec.ts:19:7 > Login @smoke > rejects a valid username with the wrong password (24.6s)
  ok 5 [firefox] > tests\login.spec.ts:27:7 > Login @smoke > protects the profile page from anonymous access (4.3s)
  ok 3 [firefox] > tests\bookstore-collection.spec.ts:80:7 > Book Store - my collection @e2e > cancelling the delete dialog keeps the book in
the collection @regression (29.9s)
  ok 7 [firefox] > tests\registration.spec.ts:32:7 > Registration @smoke > the API rejects a password that violates the documented policy
(675ms)
  ok 6 [firefox] > tests\registration.spec.ts:11:7 > Registration @smoke > the registration form is reachable from login and is captcha-
protected (6.4s)
  ok 4 [firefox] > tests\bookstore-collection.spec.ts:15:7 > Book Store - my collection @e2e > a user can register, add a book to their
collection, remove it and log out (35.0s)
  7 passed (38.1s)
Summary
  unit  passed  1.5s
  api   passed  56.6s
  ui    passed  39.1s

```

```
. - node

$ node scripts/pipeline.mjs --stage production --suite smoke --browser webkit

Local pipeline
  stage    production (BASE_URL https://demoqa.com - from default)
  suite    smoke → unit → api → ui
  browser  webkit
$ node C:\Users\g6k78\qa-assessment\playwright\node_modules\vitest\vitest.mjs run --coverage
(in .\playwright)
RUN v4.1.11 C:/Users/g6k78/qa-assessment/playwright
  Coverage enabled with v8
✓ src/utlils/test-data.test.ts (7 tests) 9ms
Test Files 1 passed (1)
Tests      7 passed (7)
Start at   11:08:29
Duration   403ms (transform 43ms, setup 0ms, import 70ms, tests 9ms, environment 0ms)
JUNIT report written to C:/Users/g6k78/qa-assessment/playwright/test-results/unit-junit.xml
% Coverage report from v8
===== Coverage summary =====
Statements : 100% ( 2/2 )
... 170 lines omitted ...
Karate version: 1.4.1 | env: production
=====
elapsed: 11,76 | threads: 3 | thread time: 10,53
features: 2 | skipped: 3 | efficiency: 0,30
scenarios: 2 | passed: 2 | failed: 0
=====
HTML report: (paste into browser to view) | Karate version: 1.4.1 | env: production
file:///C:/Users/g6k78/qa-assessment/karate/target/karate-reports/karate-summary.html
=====
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 12.58 s -- in bookstore.SmokeTest
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] -----
[INFO] BUILD SUCCESS
[INFO] -----
[INFO] Total time: 17.482 s
[INFO] Finished at: 2026-08-28T11:08:50+03:00
[INFO] -----
$ node C:\Users\g6k78\qa-assessment\playwright\node_modules\playwright\cli.js test --project=webkit --grep @smoke
(in .\playwright)
Running 5 tests using 4 workers
  ok 4 [webkit] > tests\login.spec.ts:27:7 > Login @smoke > protects the profile page from anonymous access (5.3s)
  ok 5 [webkit] > tests\registration.spec.ts:32:7 > Registration @smoke > the API rejects a password that violates the documented policy (360ms)
  ok 2 [webkit] > tests\registration.spec.ts:11:7 > Registration @smoke > the registration form is reachable from login and is captcha-protected (6.2s)
  ok 3 [webkit] > tests\login.spec.ts:4:7 > Login @smoke > rejects unknown credentials with a visible error and no session (8.4s)
  ok 1 [webkit] > tests\login.spec.ts:19:7 > Login @smoke > rejects a valid username with the wrong password (9.5s)
  5 passed (10.7s)
Summary
  unit  passed 1.5s
  api   passed 20.3s
  ui    passed 11.5s
```

Running it from VS Code

Two panels reach the same commands without typing anything.

Tasks — **Ctrl+Shift+P** → **Tasks: Run Task**. Thirteen entries are defined in [.vscode/tasks.json](#) :

Task	Runs
Pipeline: dev smoke (chromium)	the pull-request gate — also the default test task
Pipeline: dev regression (chromium)	full depth, one browser
Pipeline: dev regression (all browsers)	what the release stage runs in CI
Pipeline: release smoke / regression	pre-production
Pipeline: production smoke	post-deployment verification
Layer: unit / API / UI only	one layer at a time
Pipeline: custom...	prompts for stage, suite and browser, then runs it
Pipeline: custom... (dry run)	the same prompts, printing the commands instead
Docs: collect test metrics	re-runs the suites and rewrites the metrics section
Docs: regenerate testing_report.pdf	rebuilds the PDF

Pipeline: dev smoke is the default test task, so `Ctrl+Shift+P` → **Tasks: Run Test Task** starts it directly. *Pipeline: custom...* uses VS Code's `pickString` inputs, so the three parameters arrive as dropdowns rather than remembered flags.

NPM Scripts — the panel in the Explorer sidebar (enable with `"npm.enableScriptExplorer": true` if you do not see it) lists every script in the root `package.json`: `dev:smoke`, `release:regression`, `production:smoke`, `test:unit`, and so on. Clicking one runs it in a terminal. Scripts from `playwright/package.json` appear there too, under that folder.

Both routes end up at the same `scripts/pipeline.mjs`, so a task, an npm script and a hand-typed command produce identical runs.

One detail in the task definitions worth keeping if you edit them: every task is `"type": "process"`, not `"shell"`. VS Code launches shell tasks with your default shell, so on Windows a `shell` task would hand `--grep @smoke` to PowerShell and the tag would vanish — the same bug documented above. `process` passes the argument array straight to the executable, with nothing in between.

Why a runner rather than raw commands

Two bugs in this repository came from shells rather than from tests, and the runner exists so neither can happen again.

- `--grep @smoke` **silently loses its argument in PowerShell**. `@` is the splatting operator, so a bare `@smoke` expands the undefined variable `$smoke` to nothing and Playwright reports `"option '-g, --grep' argument missing"` — an error that names Playwright for the shell's doing.
- **Running from the wrong directory** makes `npm` download a throwaway copy of Playwright, which then finds no config and reports `Project(s) "chromium" not found. Available projects: ""`.

The runner spawns every command with an argument array and **no shell**, so nothing is re-parsed and no quoting rule applies. Paths resolve from the script's own location rather than the working directory, so it behaves the same from the root or a subfolder. Inputs are checked against allow-lists before they reach a command line:

```
--stage staging      → must be one of: dev, release, production
--browser safari     → must be one of: chromium, firefox, webkit, msedge, all
```

The network-bound layers get one retry and a per-layer timeout, because demoqa.com has been measured answering a login in 25–30s and one slow response should not read as a red suite.

Running the real workflow with `act`

`act` executes `ci.yml` itself in a container, which is closer to a GitHub runner than any wrapper can be. It needs Docker, so it is the optional route; [.github/act/](#) holds the event payloads and `.actrc` pins an image that already has Java and Node:

```
act workflow_dispatch -W .github/workflows/ci.yml -e .github/act/dev-smoke.json
act workflow_dispatch -W .github/workflows/ci.yml -e .github/act/release-regression.json
act workflow_dispatch -W .github/workflows/ci.yml -e .github/act/production-smoke.json
```

Two cautions. `act` runs the promotion jobs too, and those execute real `git push` commands — every event file here sets `promote: false` for that reason. And it runs the full browser matrix in containers, so a regression configuration is considerably slower than the local runner.